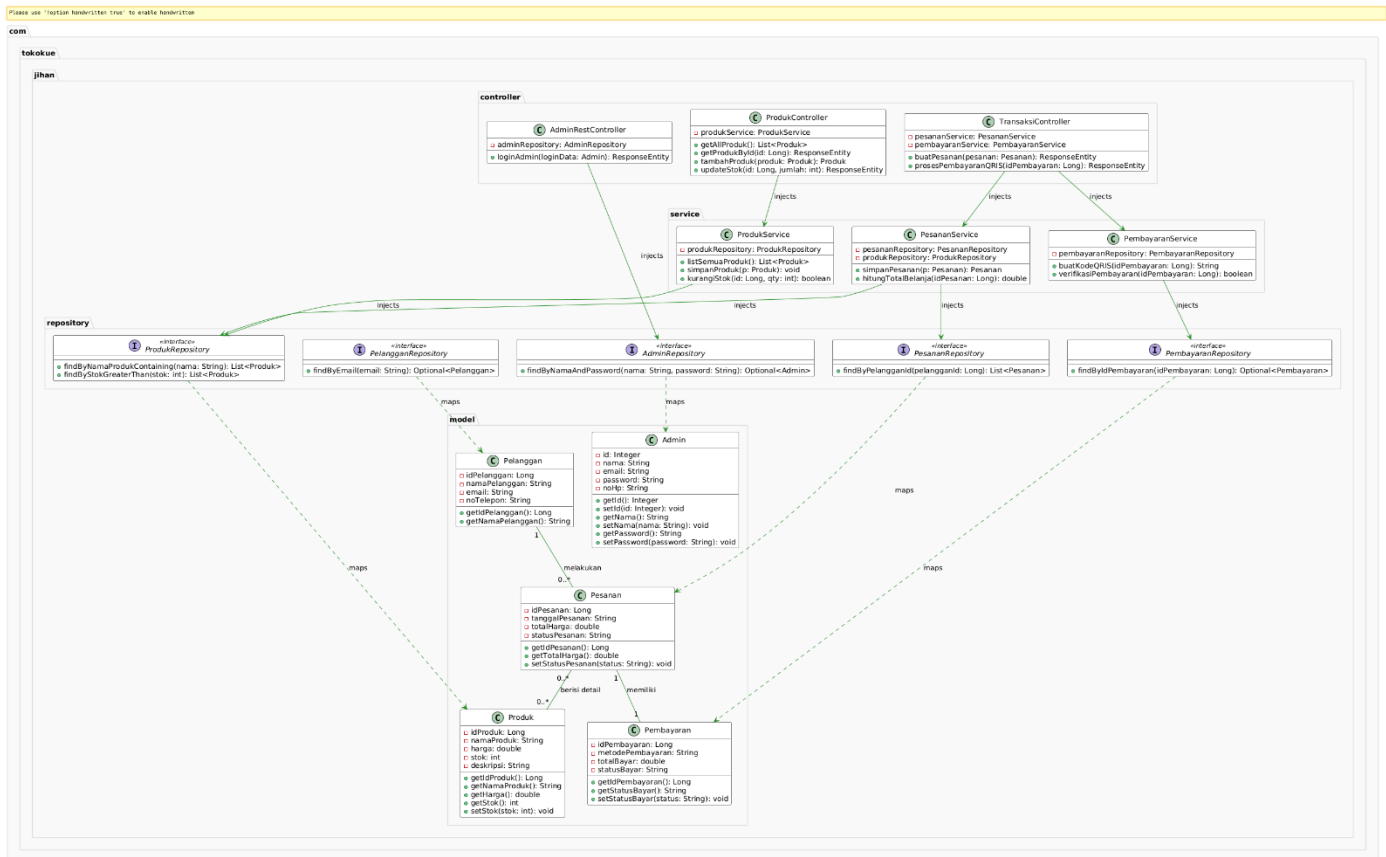


CLASS DIAGRAM

1. Skema Class Diagram

Visualisasi *class diagram* berikut menjabarkan struktur dan cetak biru komponen *backend* pada ekosistem Toko Kue Jihan. Perancangan ini membagi logika aplikasi ke dalam empat lapisan utama berbasis framework Spring Boot, yakni entitas data (Model), abstraksi basis data (Repository), logika bisnis (Service), serta manajemen *endpoint* API (Controller).



2. Penjelasan Lengkap Class Diagram Proyek

Struktur kode pada proyek Toko Kue ini menerapkan pola **Layered Architecture** (arsitektur berlapis) yang umum dipakai pada ekosistem Spring Boot. Setiap kelas dikelompokkan ke dalam paket (*packages*) tertentu berdasarkan fungsinya demi memenuhi prinsip *Separation of Concerns* (SoC), sehingga kode tetap modular, rapi, dan mudah dirawat. Penjelasan dari tiap komponen lapisan dijabarkan sebagai berikut:

A. Lapisan Model

Lapisan ini menampung objek data bisnis (POJO / *Plain Old Java Object*) yang dipetakan secara langsung menjadi tabel database lokal H2 melalui anotasi JPA/Hibernate seperti `@Entity` dan `@Table`.

- **Admin:** Berfungsi mandiri untuk menyimpan kredensial data kelompok (Bintang, Arif, Raffy) berupa data id, nama, email, password, dan noHp. Sesuai dengan hasil pembaruan sistem terbaru, kelas Admin ini dibuat berdiri sendiri (*independent entity*) untuk mempercepat performa autentikasi tanpa membutuhkan join tabel eksternal yang rumit.
- **Produk, Pelanggan, Pembayaran, Pesanan:** Entitas lain yang merepresentasikan pengelolaan entitas toko kue, seperti pencatatan data stok kue, data transaksi pelanggan, serta konfirmasi pembayaran otomatis (seperti pembuatan notifikasi QRIS).

B. Lapisan Repositori

Seluruh kelas pada paket ini berbentuk antarmuka (*interface*) yang meng-extend `JpaRepository` dari Spring Data JPA. Operasi database dasar (CRUD) dikelola otomatis oleh framework tanpa perlu menulis kueri SQL manual.

- **AdminRepository:** Menyediakan kueri turunan khusus seperti `findByNamaAndPassword(String nama, String password)` yang menjadi fondasi utama dalam memverifikasi keabsahan akun admin saat proses *login matching* berlangsung di backend.

C. Lapisan Controller (API View Layer)

Lapisan terluar yang bertindak sebagai gerbang komunikasi utama antara tampilan *front-end* (HTML/JavaScript Fetch API) dengan sistem database backend.

- **AdminRestController:** Ditandai dengan anotasi `@RestController`. Kelas ini menangani masuknya data HTTP Request dari form login admin, membaca data kiriman JSON dari browser via `@RequestBody`, memanggil `AdminRepository` untuk mencocokkan data, dan mengembalikan respons terstruktur berupa `ResponseEntity` dengan status HTTP yang sesuai (seperti 200 OK untuk login sukses atau 401 Unauthorized jika password salah).

3. Analisis Mekanisme Interaksi dan Pola Relasi Antar-Komponen

Berdasarkan visualisasi diagram kelas yang dirancang, hubungan antar-komponen di dalam sistem Toko Kue Jihan ini diklasifikasikan ke dalam dua kategori interaksi utama, yaitu tata kelola dependensi antar-lapisan (*architectural dependencies*) dan pemetaan relasi data objek:

A. Alur Dependensi Searah dan Loose Coupling (Dependency Injection)

Sistem ini mengadopsi prinsip *Loose Coupling* (keterikatan longgar) memanfaatkan fitur *Dependency Injection* bawaan dari framework Spring Boot. Alur ketergantungan antar-kelas dirancang mengalir secara linier dari lapisan terluar menuju lapisan terdalam dengan karakteristik sebagai berikut:

- **Controller Berorientasi Service & Repository:**
AdminRestController mengandalkan AdminRepository secara langsung untuk memangkas birokrasi kueri pada proses pencocokan akun admin yang membutuhkan respon cepat (*low latency*). Sementara itu, PembayaranController mendelegasikan seluruh pemrosesan data transaksional kepada PembayaranService.
- **Enkapsulasi Logika Bisnis pada Layer Service:**
PembayaranService bertindak sebagai mediator yang mengisolasi komponen Controller dari akses database langsung. Kelas ini memanggil PembayaranRepository hanya ketika data siap untuk dieksekusi atau ketika modul eksternal membutuhkan pembuatan string kode otomatis (seperti penyiapan data enkripsi notifikasi QRIS).
- **Efisiensi Debugging:** Pola delegasi satu arah dari atas ke bawah ini (Controller → Service → Repository) memastikan bahwa jika terjadi kegagalan sistem misalnya kegagalan *fetch* autentikasi atau kesalahan pembuatan status bayar proses pelacakan kesalahan (*debugging*) dapat dilokalisasi pada layer tertentu tanpa risiko merusak fungsionalitas komponen lainnya.

B. Isolasi dan Independensi Modul pada Lapisan Model

Pada lapisan entitas (*Model Layer*), rancangan kelas sengaja didesain untuk memisahkan domain administratif dengan domain transaksional pelanggan:

- **Kemandirian Entitas Admin:** Model Admin dibuat berdiri sendiri secara penuh (*independent entity*) tanpa memiliki hubungan *foreign key* atau relasi struktural ke model Pembayaran. Pendekatan arsitektur ini diambil setelah mengevaluasi struktur database terdahulu, dengan tujuan untuk mengeliminasi beban join tabel yang tidak diperlukan saat proses pemeriksaan kredensial (login) dilakukan oleh backend.
- **Fokus Data Transaksi Pembayaran:** Model Pembayaran diisolasi agar hanya berfokus pada atribut esensial transaksi seperti *idPembayaran*, *metodePembayaran*, *totalBayar*, dan *statusBayar*. Pemisahan ini menjaga integritas data riwayat stok toko kue tetap konsisten dan aman di dalam database H2 (file-based) meskipun server mengalami *restart* berulang kali.